

Lagrange-Lock

Taming the Three-Body Problem with Reinforcement Learning

Shreyash Rai (241020269)
Sameer Choudhary (241020467)

Date: 13th February 2026
Supervisor name: Dr. Avantika Singh

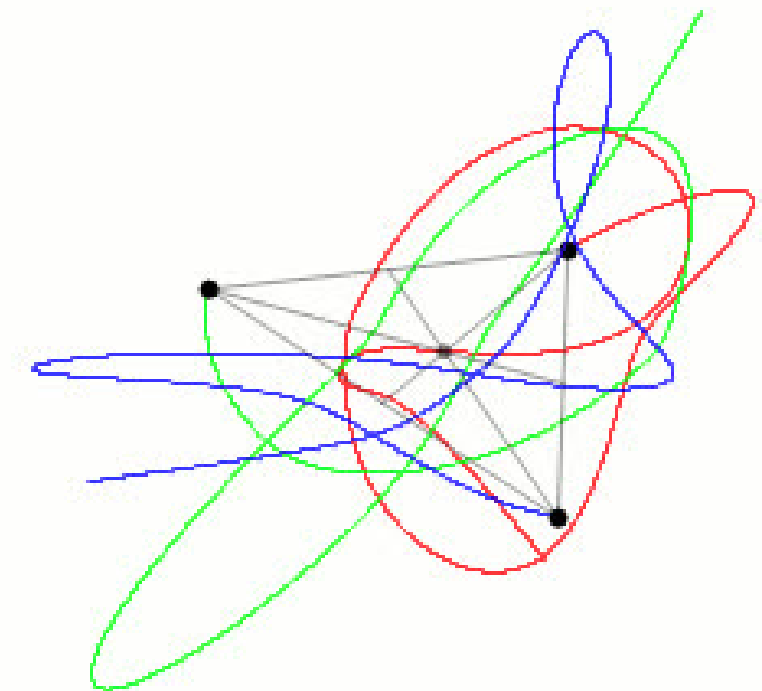


**Dr. Shyama Prasad Mukherjee International
Institute of Information Technology, Naya
Raipur**

What is the Three-Body Problem?

In physics, specifically classical mechanics, the **three-body problem** is to take the **initial positions** and **velocities** (or momenta) of **three point masses** orbiting each other in space and then to calculate their subsequent trajectories using **Newton's laws of motion** and **Newton's law of universal gravitation**.

- Unlike the **two-body problem**, the **three-body problem** has **no general closed-form analytic solution**.
- The differential equations that govern the movements of three gravitating bodies are **not integrable** and cannot be solved to give explicit formulas for the positions of the bodies as a function of time.
- For most initial conditions, the dynamical system for three orbiting bodies is **chaotic**, and the only way to predict their motions is to estimate them using numerical methods.

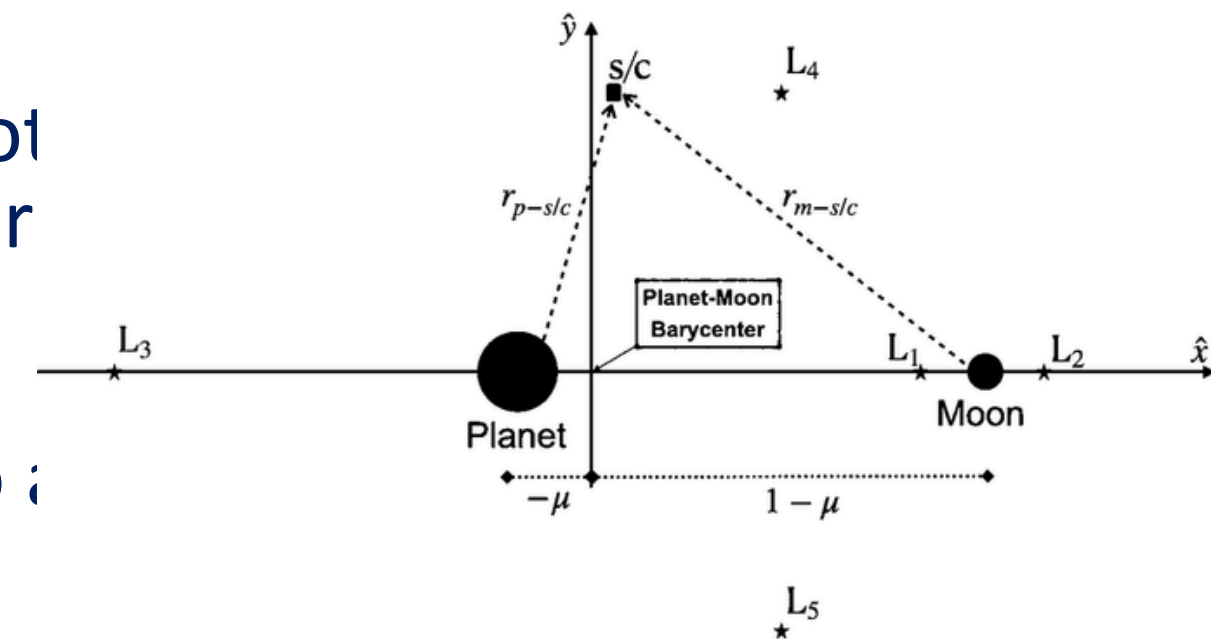


Three bodies in chaotic orbits

The CR3BP Model – Simplifying Chaos



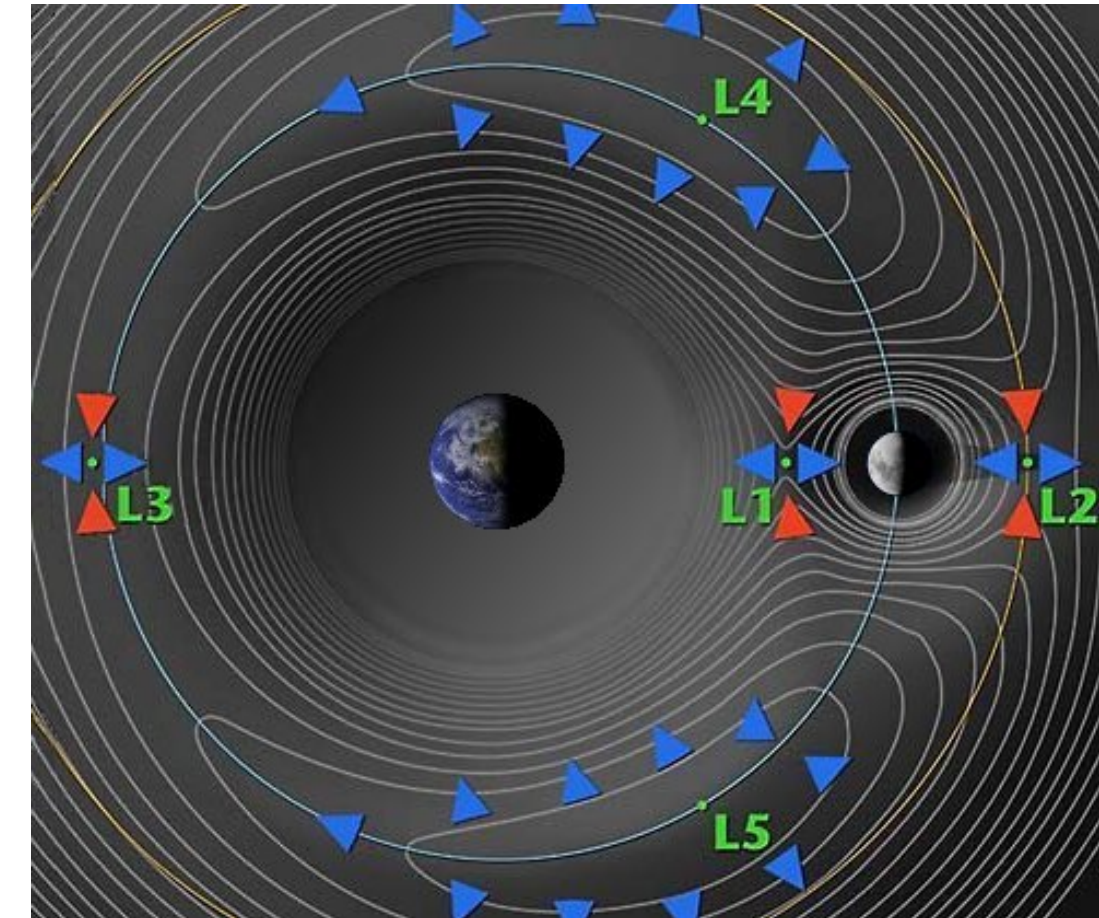
- **The Circular Restricted Three-Body Problem (CR3BP):** This model assumes two massive bodies move in perfect circles around their common center of mass, while the satellite is too small to affect their motion.
- **Normalization:** Instead of using messy units like kilometers or kilograms, we set the distance between the primaries and their total mass to 1.0. This prevents "Floating Point Errors" where computers lose precision when mixing huge and tiny numbers.
- **Mass ratio (μ):** The entire physics of the system is boiled down to a single mass ratio, μ . Changing μ allows the model to switch from Earth-Moon ($\mu \approx 0.012$) to Sun-Earth ($\mu \approx 0.000003$) instantly.
- **The Rotating Reference Frame:** We observe on a frame that spins with the planets. This makes the Lagrange points appear as stationary, fixed coordinates (x, y, z) rather than moving targets.



CR3BP Model

Lagrange Points – The Parking spots

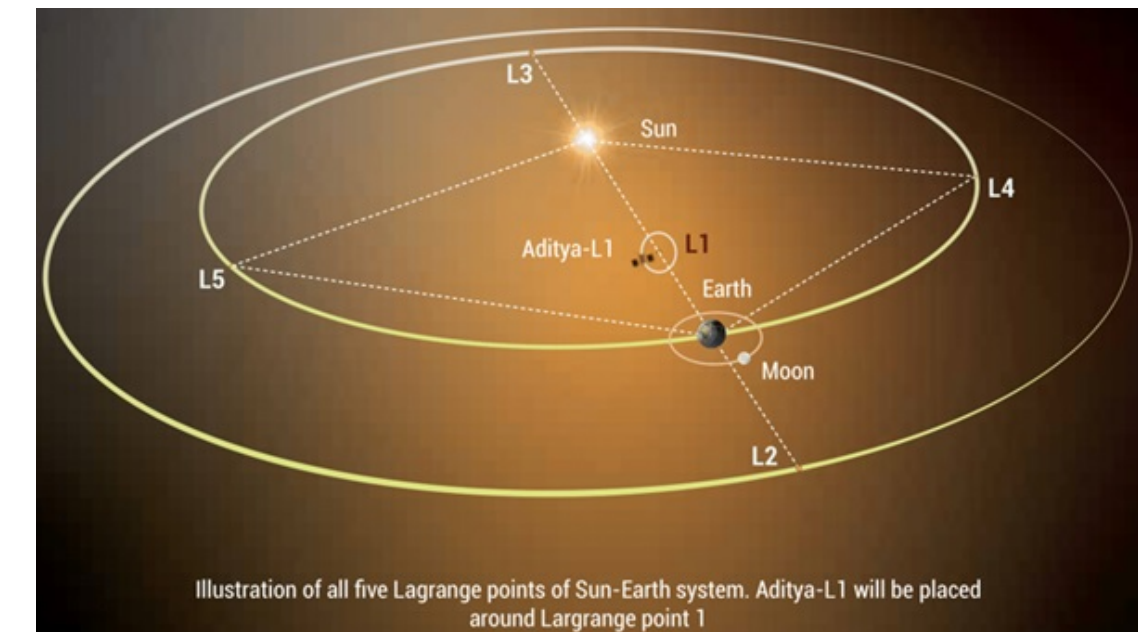
- These are **five** specific points in space where the **gravitational pull** of two large masses (like the Sun and Earth) **precisely cancels out** the **centrifugal force** felt by a smaller object.
- **The Five Points:**
 - **L1, L2, L3 (Collinear Points):** These lie on a straight line passing through the two large masses and are naturally unstable, requiring active control to stay put.
 - **L4 & L5 (Triangular Points):** These form equilateral triangles with the two masses and are naturally stable, often trapping dust or asteroids.
 -
- **Real-World Utility:**
 - **L1:** Used by solar-observing satellites like Aditya L1 because it has an uninterrupted view of the Sun.
 - **L2:** Home to the James Webb Space Telescope (JWST), using Earth to shield its sensors from the Sun's heat.



The Lagrange Points

Lagrange-Lock – Our Project

- **Goal:** To develop an autonomous system for Station-Keeping—the art of keeping a satellite in its assigned "parking spot" despite constant gravitational pull.
- **The "Universal" Solver:** By using normalized math, our project isn't just for Earth or the Moon; it is designed to work in any celestial system (e.g., Sun-Jupiter or distant exoplanetary systems) just by adjusting the mass ratio, μ .
-
- **Implicit Intelligence:** Unlike traditional software that needs a perfect map, our project builds a "Blind Pilot" that can handle sensor noise and uncertainty, making it robust for deep space exploration.
-
- **Efficiency First:** The project focuses on "Fuel-Optimal" maneuvers i.e teaching the AI to use gravity to its advantage rather than constantly fighting against it with thrusters.



Project Visualisation

The Reward Function

We calculate the reward R_t at every time step t :

$$R_t = w_{survive} \cdot R_{alive} - w_{fuel} \cdot ||A_t|| - w_{dist} \cdot D_{L1}$$

Component Breakdown:

- **R.alive (+1.0)**: A constant reward given for every time step the satellite remains within the simulation bounds (doesn't crash). This encourages the agent to simply stay alive.
- **Distance Penalty** ($-1.0 \times D_{L1}$) : A penalty proportional to the distance from the L1 point. This forces the agent to move toward the target.
- **Fuel Penalty** ($-0.1 \times ||A_t||$) : A small penalty for using thrusters. This prevents the agent from jittering and encourages "drifting" on natural manifolds.
- We are currently adjusting the Weights (w) of these terms.
- If the Distance Penalty is too high, the agent panics and crashes while trying to rush to the center. We are tuning $w_{survive}$ to be higher to prioritize stability first.



Proximal Policy Optimization (PPO)



- We chose **Proximal Policy Optimization** because it is the industry standard for continuous control problems (like robotics). It strikes a balance between ease of tuning and sample efficiency.
- **Actor-Critic Architecture:**
 - **The Actor (Policy):** This network decides which thrusters to fire (Action) based on the current state.
 - **The Critic (Value Function):** This network predicts how good that action was (Expected Reward), helping the Actor learn faster.
 - **The Mathematical Core (Clipped Objective):** To prevent the AI from making drastic, destabilizing updates, PPO "clips" the learning step:

$$L(\theta) = \min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)$$

- $r_t(\theta)$: The probability ratio (New Policy / Old Policy).
- $\hat{A}_t$: The Advantage function (Did this action do better than average?).
- ϵ : A hyperparameter (usually 0.2) that limits how much the policy can change at once.

PP0 Pseudo-Code



```
Initialize PPO_Agent(Policy_Network, Value_Network)

For Episode in 1 to Max_Episodes:
    State = Environment.Reset()

    While True:
        # 1. Actor decides thrust based on State
        Action = Agent.Select_Action(State)

        # 2. Physics Engine calculates result
        Next_State, Reward, Done = Environment.Step(Action)

        # 3. Store experience
        Agent.Memory.Store(State, Action, Reward)

        # 4. Update Networks (Optimize L_clip)
        If Time_To_Update:
            Agent.Learn()
```

Implementation & Tech Stack



- **The Hybrid Architecture:**
- We utilize a dual-language approach to balance Computational Speed (for training) with Accessibility (for demonstration).
 - **1. The Training Stack (Python):**
 - **Stable Baselines3 (SB3):** The backbone of our PPO agent implementation.
 - **Gymnasium:** Used to wrap our custom physics engine into a standard RL environment (State/Action interface).
 - **Numba & SciPy:** Used to JIT-compile the differential equations, accelerating the physics calculations to >10,000 steps per second for rapid training.
 - **2. The Visualization Stack (JavaScript):**
 - **Three.js:** A WebGL library used to render the 3D star system, satellite trajectory, and Lagrange points in the browser.
 - **Custom RK4 Engine:** A pure JavaScript port of our physics core, running client-side to verify the math without needing a backend server.
- Workflow:
- **Step 1:** Train the Agent in Python (Headless/No Graphics).
- **Step 2:** Export the model weights.
- **Step 3:** Load the weights into the JavaScript viewer for real-time demonstration.

Conclusion & Future Directions



- **Physics Verified:** We have successfully built a high-fidelity "Digital Twin" of the Three-Body Problem using the CR3BP model and RK4 integration.
- **Real-Time Visualization:** The JavaScript engine renders chaotic orbits at 60 FPS, providing immediate visual feedback on satellite behavior.
- **RL Infrastructure:** The PPO agent is fully integrated with the environment, receiving state inputs and executing thrust actions.
- **Work in Progress:**
 - Our AI agent is currently in the "Crash Phase".
 - It struggles to balance the conflicting goals of "Fuel Saving" vs. "Distance Minimization," often resulting in high-speed drift or collision.
- **Future Roadmap (Phase 3 & 4):**
 - Reward Shaping: We will refine the penalty weights to guide the agent toward stable orbits.
 - Noisy Sensors: Once stability is achieved, we will introduce Gaussian noise to train the "Blind Pilot" robustly.
 - We will disable 1-2 thrusters to see how the agent behaves in the final stage of the project.